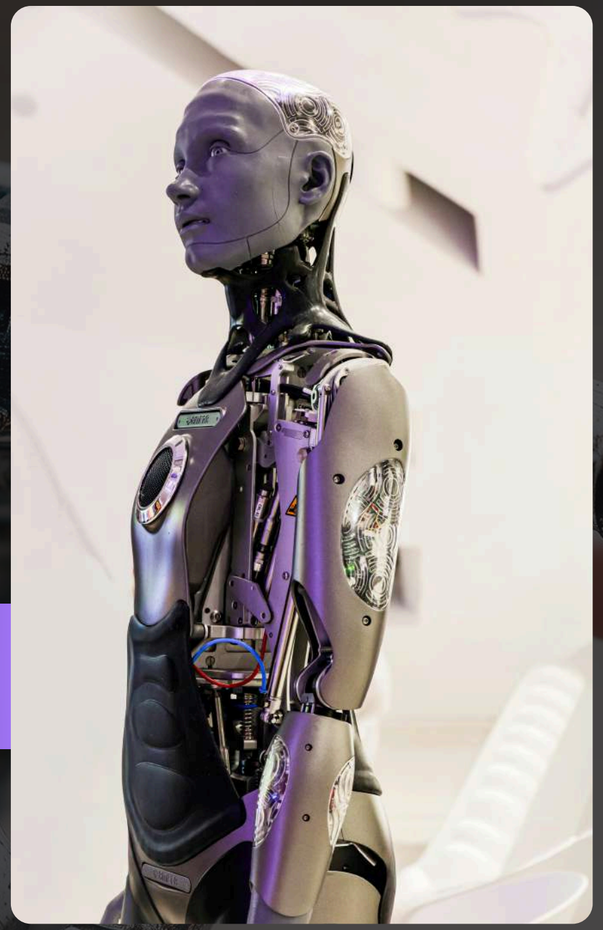


UNIT 6

Deep Learning

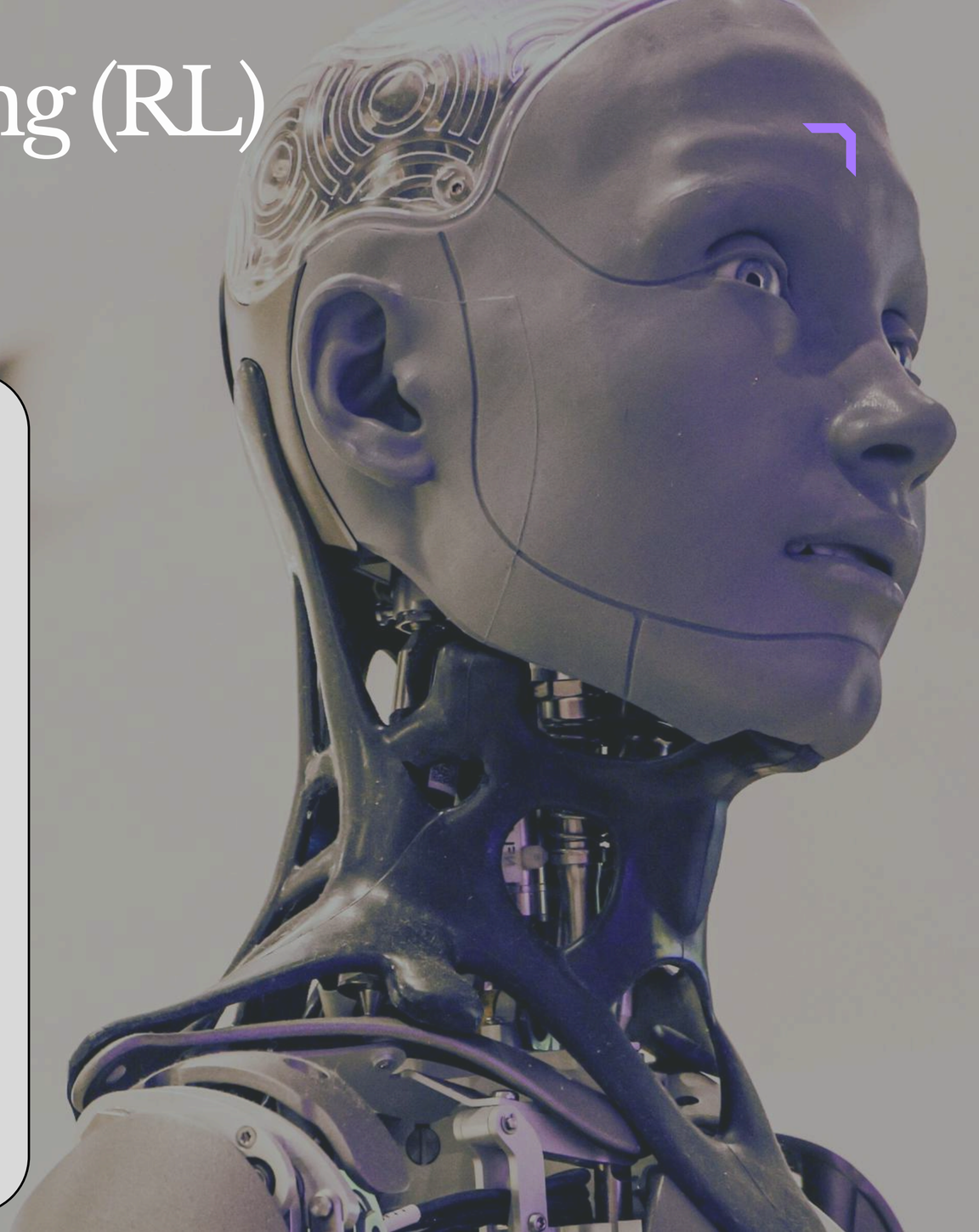
Reinforcerment Learning

Reinforcement Learning (RL)

Introduction

- Reinforcement Learning (RL) is a type of Machine Learning.
- In RL, an agent learns by interacting with the environment.
- The agent performs actions and receives rewards or penalties.
- The main goal is to maximize total reward over time.
- The learning process is inspired by how humans and animals learn from experience.

Example



Reinforcement Learning (RL)

Introduction

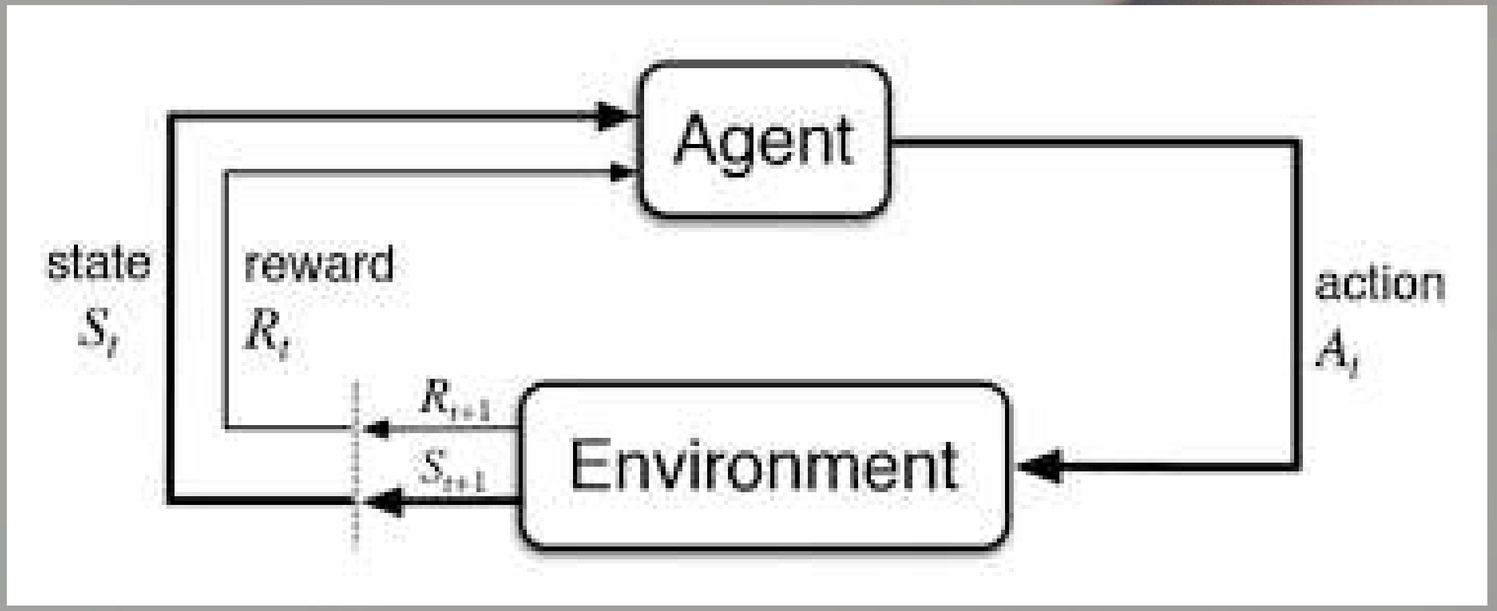
- A child learns not to touch a hot object after getting burned once.
- Similarly, an RL agent learns good and bad actions through trial and error.

Basic Components of RL

- Agent → Decision maker
- Environment → Surrounding system where agent works
- State (St) → Current situation
- Action (At) → Action performed by agent
- Reward (Rt) → Feedback received after action

Reinforcement Learning (RL)

Diagram



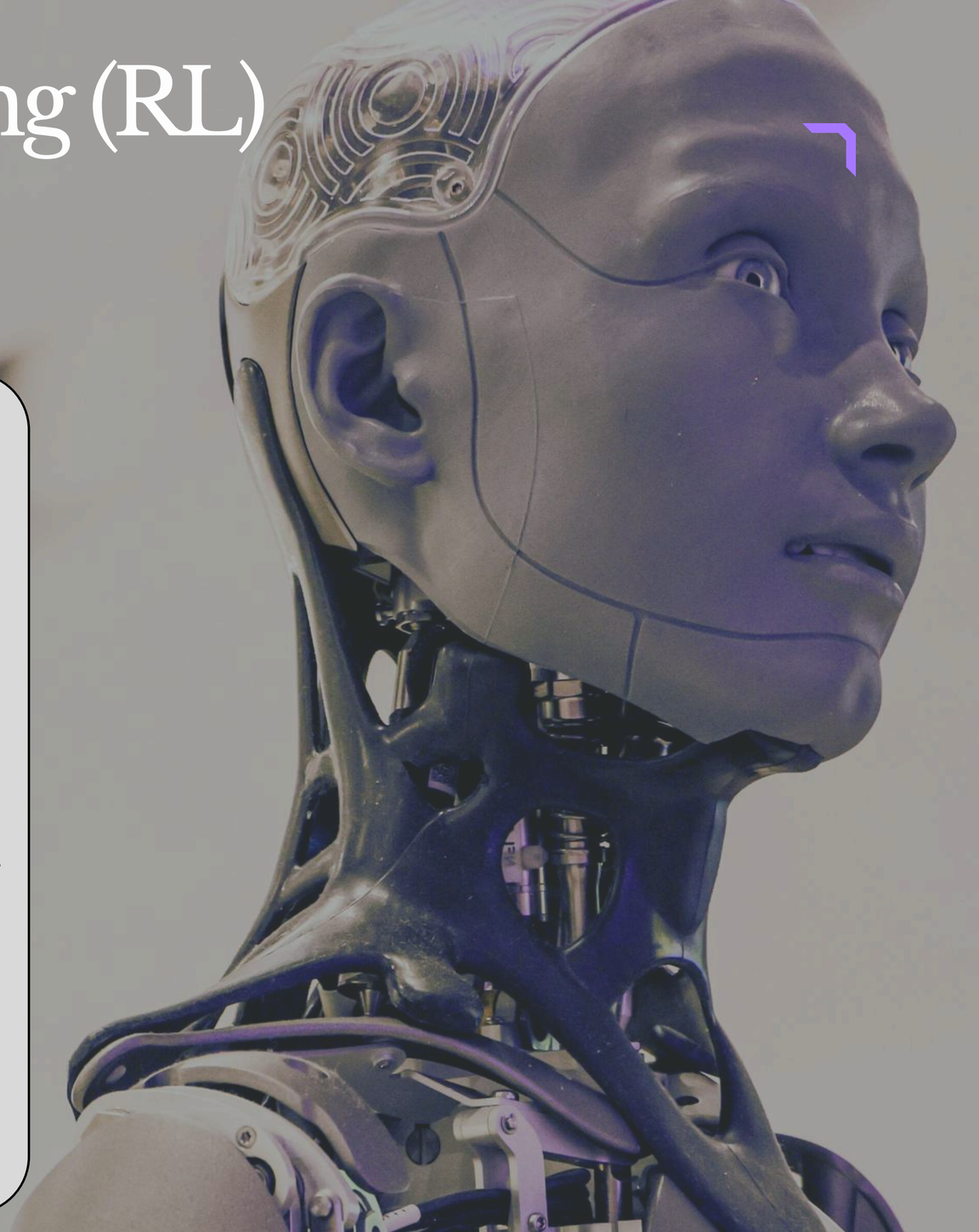
Reinforcement Learning (RL)

Advantages

1. Learns from Experience
   - RL does not require labeled data.
   - It learns using trial and error.

2. Adapts to Dynamic Environments
   - RL agents can adjust when the environment changes.
   - Useful for real-world continuously changing systems.

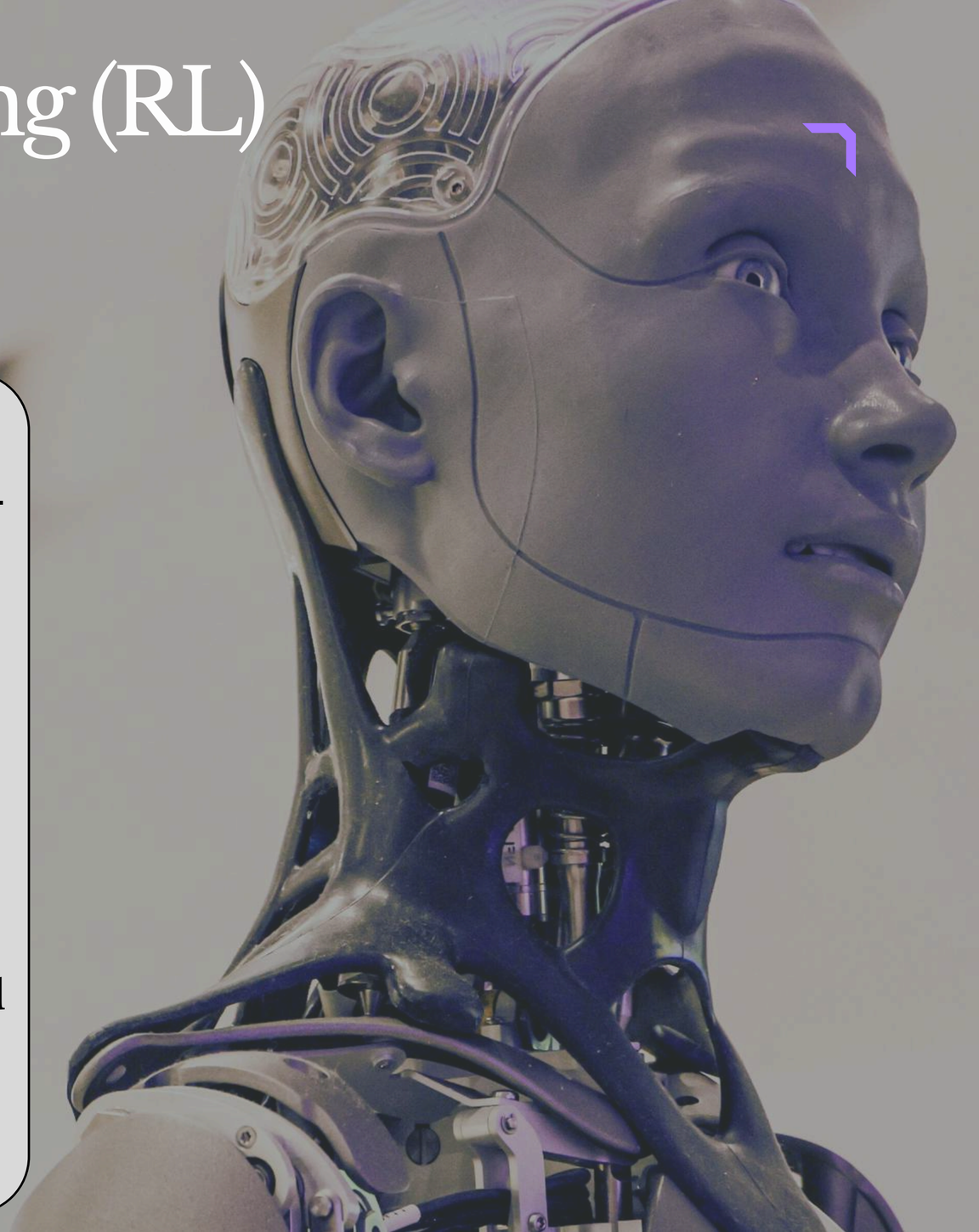
3. Solves Sequential Decision Problems
   - Suitable for tasks involving multiple decisions.
   - Used in games, robotics, self-driving cars, etc.



Reinforcement Learning (RL)

Disadvantages

1. Requires High Time and Computation
   - Training RL models can be slow and resource-intensive.
   - Complex environments increase training time.
2. Can Become Unstable or Unsafe
   - Poor reward design may lead to unwanted behavior.
   - Agent may focus only on maximizing reward.
3. Exploration Can Be Costly
   - Trying random actions can be risky in real-world systems.
   - Example: testing random actions in drones or robots.



Reinforcement Learning (RL)

Applications

- **Robotics**
- **Self-driving cars**
- **Game playing (Chess, AlphaGo)**
- **Recommendation systems**
- **Industrial automation**

Reinforcement Learning (RL)

Characteristics

1. Trial-and-Error Learning
   - RL is based on trying different actions and learning from results.
   - The agent keeps experimenting and gradually improves its performance.
   - Learning happens through rewards and punishments.

2. Exploration vs Exploitation
   - RL balances between:

Reinforcement Learning (RL)

Characteristics

- Exploration → Trying new actions to discover better rewards.
- Exploitation → Using known actions that already give high rewards.

- This balance is important for efficient learning.

3. Delayed Rewards

- Rewards in RL are often not immediate.
- The agent may receive the actual reward after several actions.

Reinforcement Learning (RL)

Characteristics

- It learns to focus on long-term success instead of short-term gain.

4. Learning from Interaction

- Unlike Supervised Learning, RL learns by interacting with the environment.
- The agent:
  - Observes the current state
  - Takes an action
  - Receives the next state and reward

Reinforcement Learning (RL)

Characteristics

5. Environment Dependent

- **The learning process depends heavily on the environment and its rules.**
- **The same algorithm may behave differently in different environments.**

6. Policy and Value Function

- **Policy**
- **A policy defines how an agent chooses actions in different states.**

Reinforcement Learning (RL)

Characteristics

Value Function

- **A value function estimates how good a state or action is.**
- **It predicts expected future rewards.**

Reinforcement Learning (RL)

Types of RL

1. Positive Reinforcement Learning

- **Positive RL means rewarding the agent when it performs the correct action.**
- **This encourages the agent to repeat the same action in the future.**
- **The goal is to maximize rewards.**

Example

- **In a game, giving points to a player for making a correct move.**

Reinforcement Learning (RL)

Types of RL

2. Negative Reinforcement Learning

- **Negative RL means removing something unpleasant when the agent performs the correct action.**
- **It helps the agent avoid bad situations and improve performance.**

Example

- **In self-driving cars, reducing penalties when the car stays in the correct lane.**

Reinforcement Learning (RL)

Types of RL

3. Model-Based Reinforcement Learning

- **In Model-Based RL, the agent builds a model of the environment.**
- **The model predicts:**
  - **Next state**
  - **Expected reward after an action**
- **The agent plans actions using these predictions.**

Example

- **A robot simulating different movements before performing them.**

Reinforcement Learning (RL)

Types of RL

4. Model-Free Reinforcement Learning

- **In Model-Free RL, the agent does not create a model of the environment.**
- **It learns directly from actions and rewards.**
- **The agent remembers successful actions and uses them again.**

Example

- **A robot learning paths by trial and error without knowing the map.**

Reinforcement Learning (RL)

Types of RL

5. Online Reinforcement Learning

- **In Online RL, the agent learns in real-time while interacting with the environment.**
- **The strategy is continuously updated using new experiences.**

Example

- **Chatbots improving responses through user interactions.**

Reinforcement Learning (RL)

Types of RL

6. Offline (Batch) Reinforcement Learning

- **In Offline RL, the agent learns from a fixed dataset of past experiences.**
- **It does not interact with the environment during training.**

Example

- **Training AI systems using previously collected driving data.**

Reinforcement Learning (RL)

Challenges in RL

1. Exploration vs Exploitation Dilemma

- **One of the biggest challenges in RL is deciding whether to explore or exploit.**

Exploration

- **Trying new actions to discover better rewards.**

Exploitation

- **Using known actions that already give high rewards.**
- **If the agent explores too much, it may waste time on poor actions.**
- **If it exploits too early, it may miss better strategies.**

Reinforcement Learning (RL)

Challenges in RL

2. Delayed Rewards

- **In RL, rewards are often not given immediately.**
- **The agent may need to perform several steps before receiving feedback.**
- **This makes it difficult to identify which action caused the reward or penalty.**

Example

- **In chess, the reward (win/loss) is received only at the end of the game.**
- **This makes learning slow and complex.**

Reinforcement Learning (RL)

Challenges in RL

3. Large or Continuous State Spaces

- **Many RL problems involve environments with:**
  - **Huge numbers of states**
  - **Continuous variables like speed, position, etc.**
- **The agent cannot visit and remember every possible state.**
- **This creates challenges in:**
  - **Training**
  - **Storage**
  - **Decision making**

Reinforcement Learning (RL)

Challenges in RL

4. Sample Inefficiency

- **RL often requires a large number of training episodes.**
- **The agent learns through repeated trial and error.**
- **Training can become time-consuming and expensive.**

Example

- **Training robots in the real world may cause wear and tear or costly mistakes.**

Deep Reinforcement Learning (Deep RL)

Introduction

- Deep Reinforcement Learning is a combination of:
  - Reinforcement Learning (RL)
  - Deep Learning
- In Deep RL, agents use deep neural networks instead of simple tables or formulas.

- The neural network helps the agent learn:
  - Best actions
  - Best strategies
  - Value of actions
-

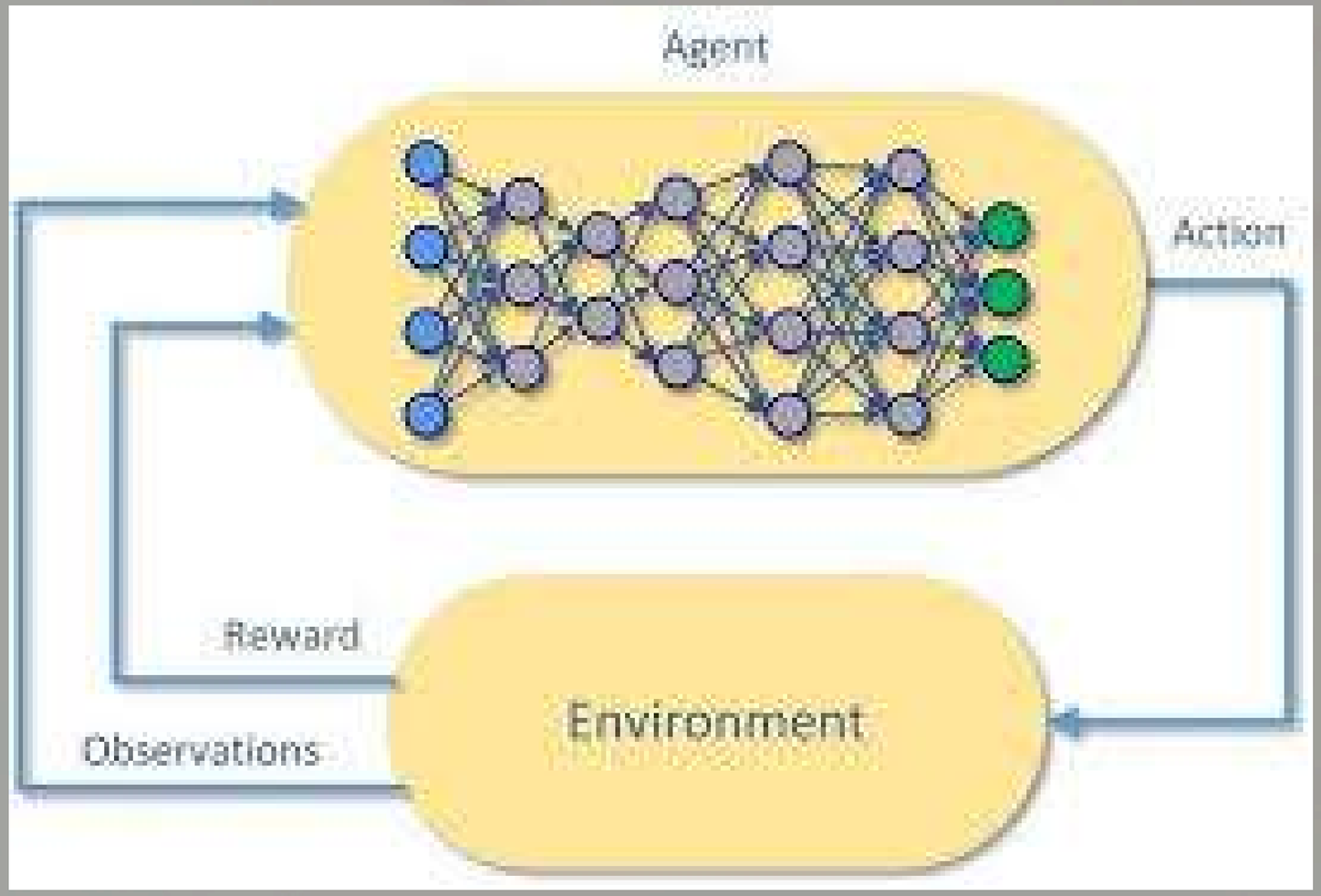
Deep Reinforcement Learning (Deep RL)

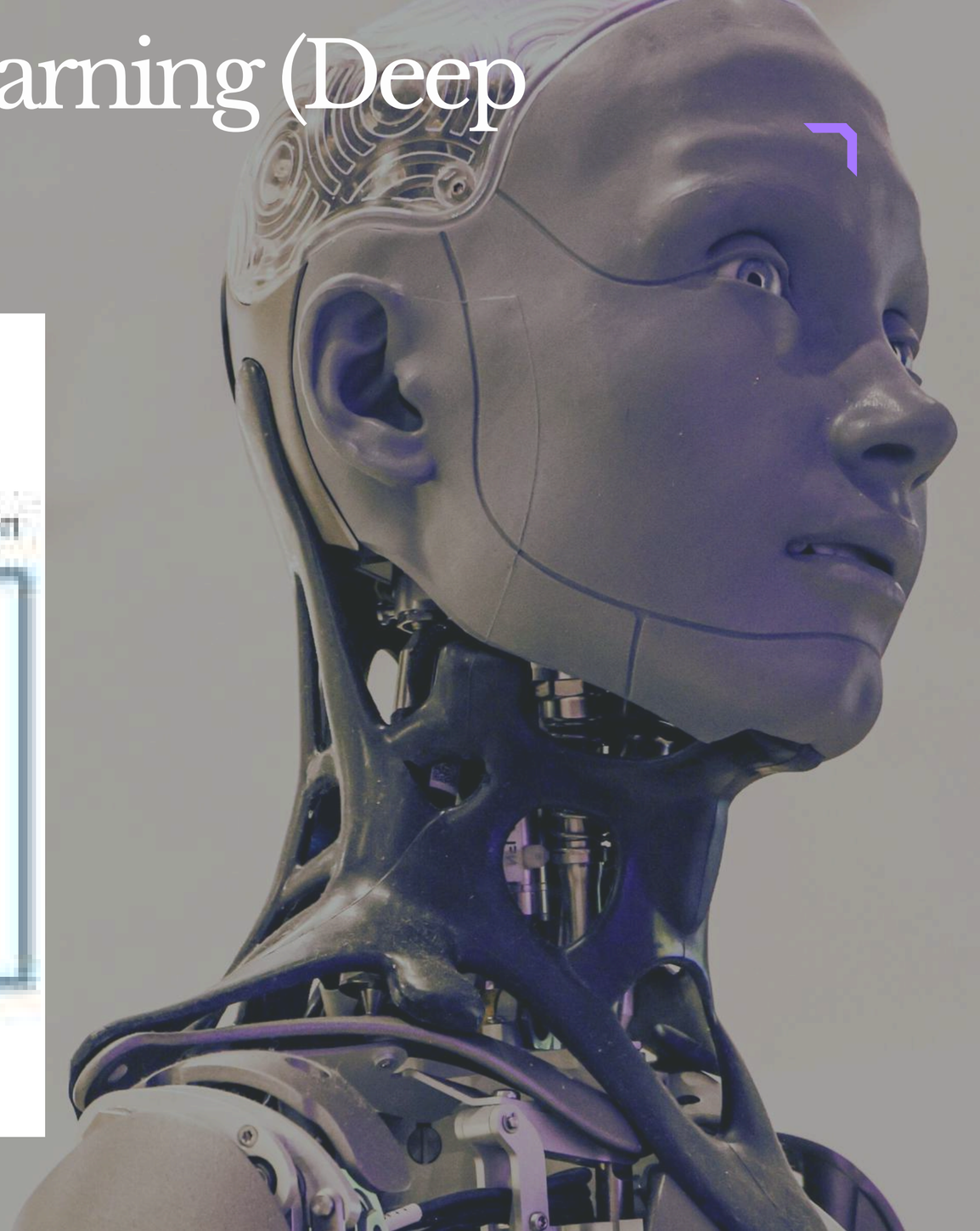
Introduction

- Deep RL can handle complex environments with high-dimensional data such as:
- Images
- Videos
- Sensor data

Diagram

Deep Reinforcement Learning (Deep RL)





Deep Reinforcement Learning (Deep RL)

Working of Deep RL

Step 1: Observe Environment

- The agent observes the environment.
- Input may include images or sensor data.

Step 2: Neural Network Processing

- A deep neural network processes the input data.
- The network predicts the best action.

Step 3: Perform Action

- The agent performs the selected action in the environment.

Deep Reinforcement Learning (Deep RL)

Working of Deep RL

Step 4: Receive Reward

- The environment gives feedback in the form of rewards or penalties.

Step 5: Update Network

- The neural network updates itself using the received feedback.
- Over time, the agent improves through continuous learning.

Deep Reinforcement Learning (Deep RL)

Example

Self-Driving Cars

- Self-driving cars use Deep RL to learn safe navigation.
- The system learns by interacting with:
  - Simulations
  - Real-world driving environments

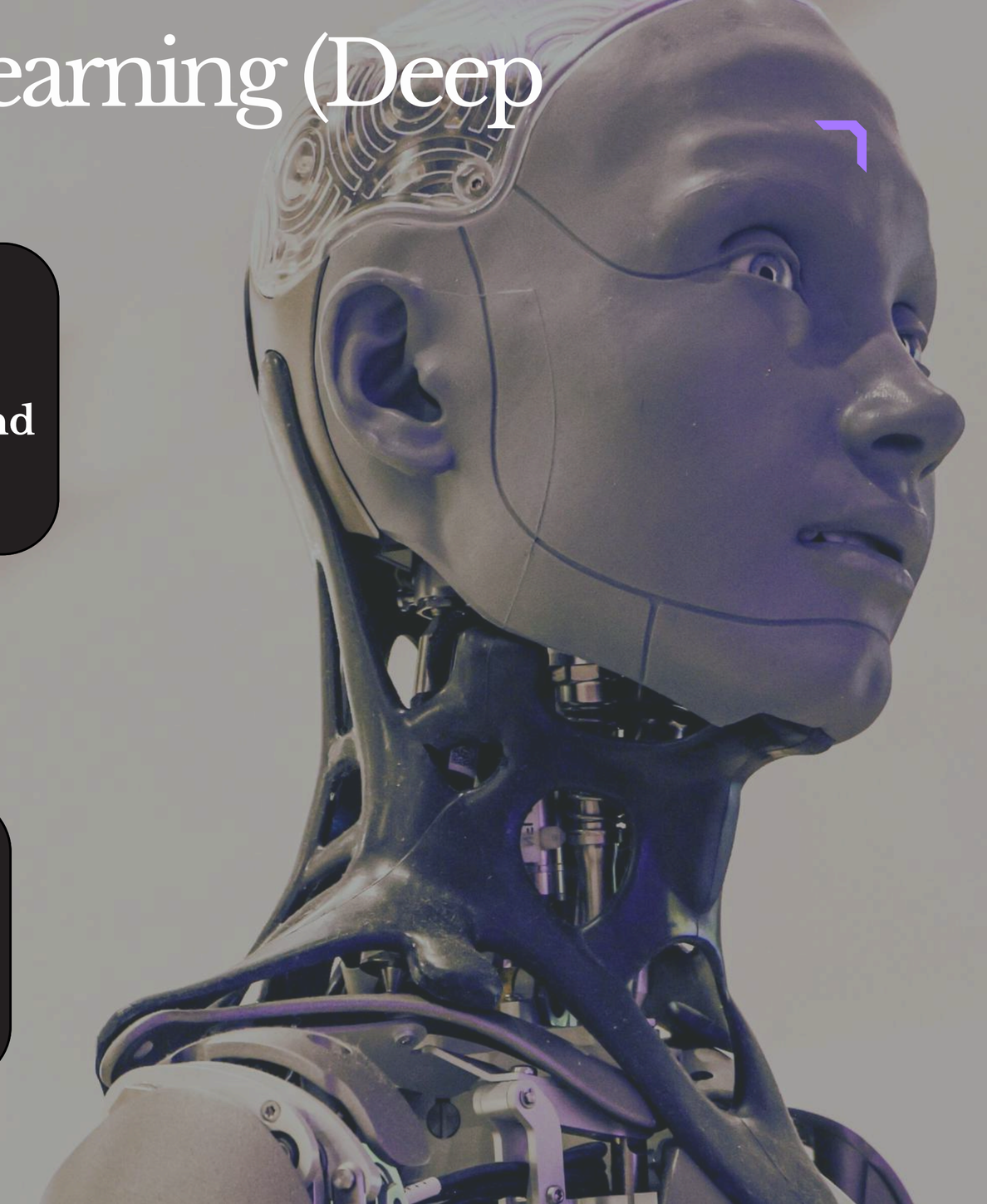
Advantages

Deep Reinforcement Learning (Deep RL)

- Handles large and complex data efficiently.
- Learns automatically from experience.
- Useful in robotics, gaming, healthcare, and autonomous systems.

Disadvantages

- Requires large amounts of data.
- Needs high computational power for training.
- Training can be slow and expensive.



Markov Decision Process (MDP)

Introduction

- A Markov Decision Process (MDP) is a mathematical framework used to describe decision-making situations.
- In MDP, outcomes are:
  - Partly controlled by the agent
  - Partly random
- MDP helps determine how an agent should act to maximize long-term rewards.
- It assumes that the future depends only on the current state and not on past states.

Markov Decision Process (MDP)

Components of MDP

1. States (S)
   - States represent all possible situations or configurations of the agent.
   - Example
   - In a grid game, each cell can represent a state.

2. Actions (A)
   - Actions are the choices available to the agent in each state.

Example
   - Move down

Markov Decision Process (MDP)

Components of MDP

- Move left
- Move right
- Move up

3. Transition Probability (P)

- Transition probability defines the probability of moving from one state to another after taking an action.

Example

- A robot moving forward may reach a new position with some probability.

Markov Decision Process (MDP)

Components of MDP

4. Reward (R)

- Reward is the feedback received after performing an action.
- The goal of the agent is to maximize total reward over time.
- Example
- A robot receives +10 reward for reaching the charging station.

5. Policy (π)

Markov Decision Process (MDP)

Components of MDP

- A policy is the strategy followed by the agent.
- It maps states to actions.

Example

- If obstacle ahead → turn right.

Markov Property

- The Markov Property states:

“The next state depends only on the current state and action, not on previous events.”

- The agent does not need complete history to make decisions.
- Only the current state is important.

Markov Decision Process (MDP)

Example of MDP

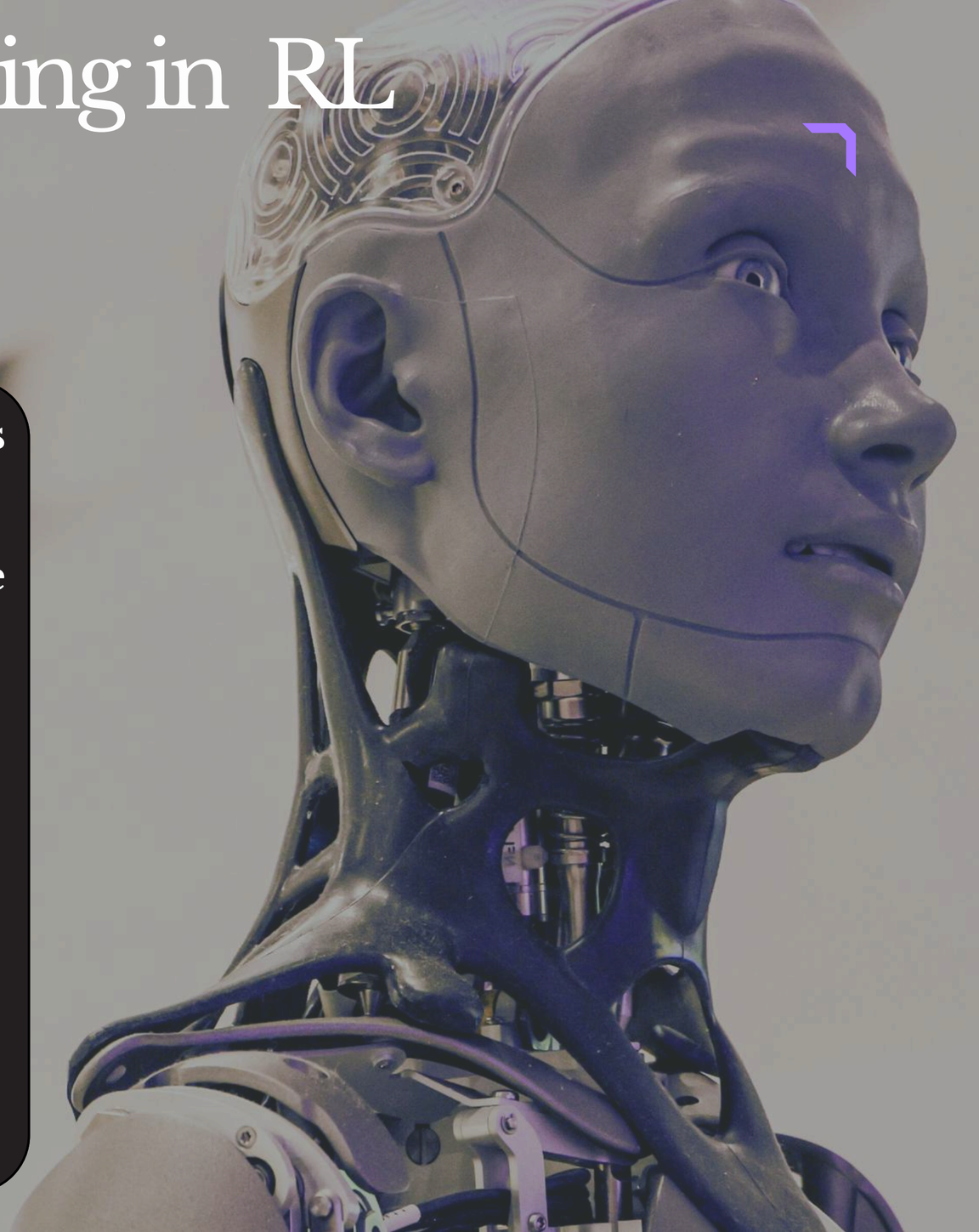
Robot Navigation

- State
- Current location of the robot.
- Actions
- Move forward
- Turn left
- Turn right
- Transition
- Moving forward changes the robot's position with certain probability.

Dynamic Programming in RL

Introduction

- Dynamic Programming (DP) is a set of algorithms used to solve Reinforcement Learning problems.
- DP is used when we have a complete and accurate model of the environment.
- The model includes:
- States
- Actions
- Rewards
- Transition probabilities
- DP helps in finding:



Dynamic Programming in RL

Introduction

- Optimal policy (best actions)
- Value functions (how good a state is)
- DP solves problems by:
- Breaking them into smaller subproblems
- Solving each part
- Combining the results

Various DP Algorithms in RL

Deep Reinforcement Learning (Deep RL)

1. Policy Evaluation

- Policy Evaluation computes the value function V(s) for a given policy π\piπ.
- It repeatedly updates state values using the Bellman Expectation Equation.
- $V(s) = \Sigma a \pi(a \mid s) \Sigma s', rp(s', r \mid s, a)[r + \gamma V(s')]$
- The process continues until the values converge.
- It helps determine how good a fixed policy is.

2. Policy Improvement

- After finding the value function, the policy can be improved.

Deep Reinforcement Learning (Deep RL)

- The agent chooses actions that lead to higher rewards.
- This process updates the policy greedily.

Goal

- Select the best possible action in each state.

3. Policy Iteration

- Policy Iteration combines:
  - Policy Evaluation
  - Policy Improvement

Steps

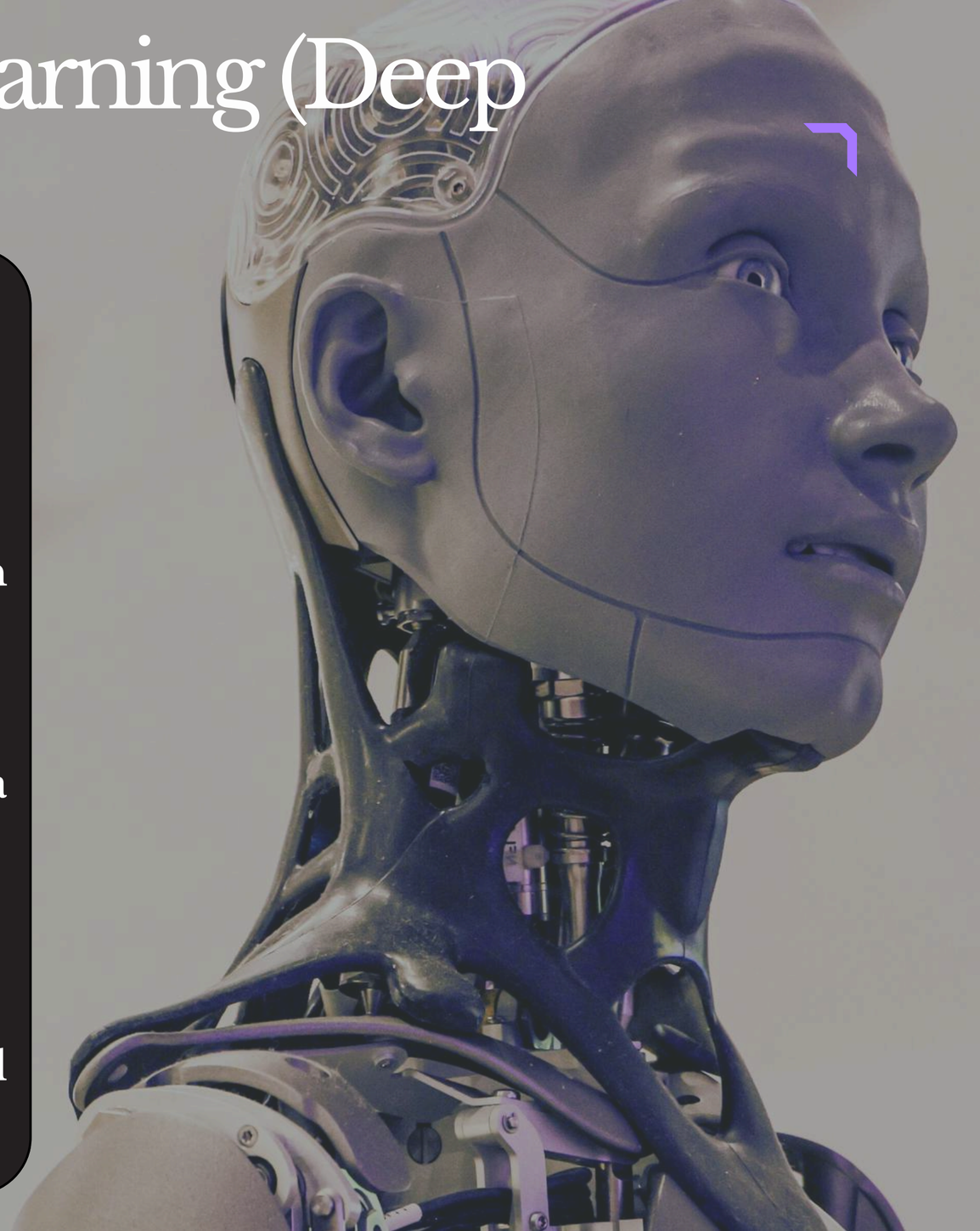
1. Evaluate the current policy.
- Improve the policy using updated values.

Deep Reinforcement Learning (Deep RL)

- Repeat until the policy no longer changes.
- This method helps find the optimal policy.

4. Value Iteration

- Value Iteration directly updates the value function using the Bellman Optimality Equation.
- $V(s)=\max_a \Sigma_{s',r} p(s',r \mid s,a)[r+\gamma V(s')]$
- It combines evaluation and improvement into a single step.
- Often faster and more efficient than Policy Iteration.
- Result
- Once the value function converges, the optimal policy can be derived.

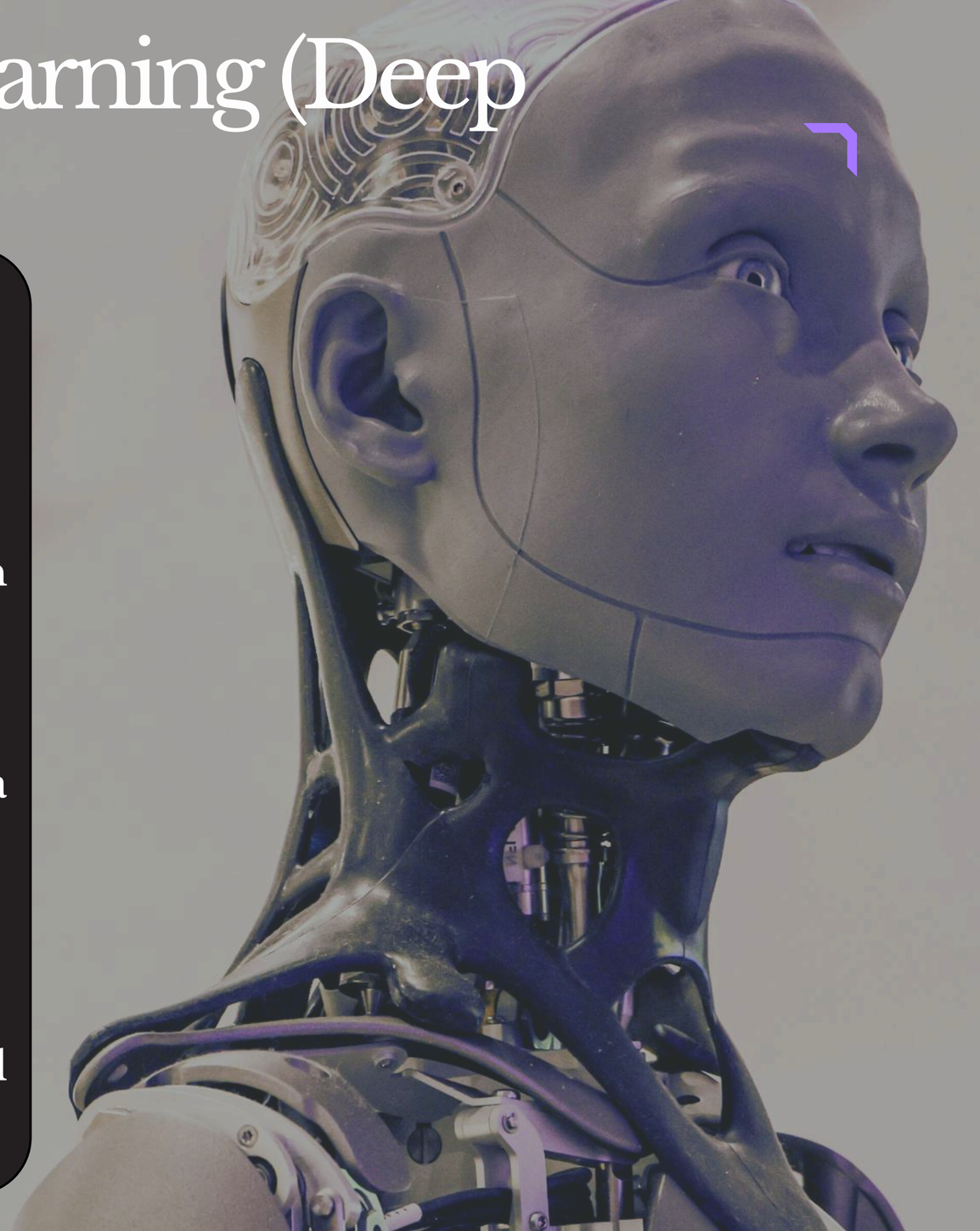


Deep Reinforcement Learning (Deep RL)

- Repeat until the policy no longer changes.
- This method helps find the optimal policy.

4. Value Iteration

- Value Iteration directly updates the value function using the Bellman Optimality Equation.
- $V(s)=\max_a \Sigma_{s',r} p(s',r \mid s,a)[r+\gamma V(s')]$
- It combines evaluation and improvement into a single step.
- Often faster and more efficient than Policy Iteration.
- Result
- Once the value function converges, the optimal policy can be derived.



Q learning and Deep Networks

Q-Learning

- Q-Learning is a type of Model-Free RL algorithm.
- It helps an agent to learn how to act optimally in an environment by learning quality (Q-value) of taking an action in a particular state without knowing full environment in advance.
- It works by updating Q-values using Bellman equation.
- The Q-value tells the agent how good it is to take a specific action from a given state, aiming to maximize total reward over time.

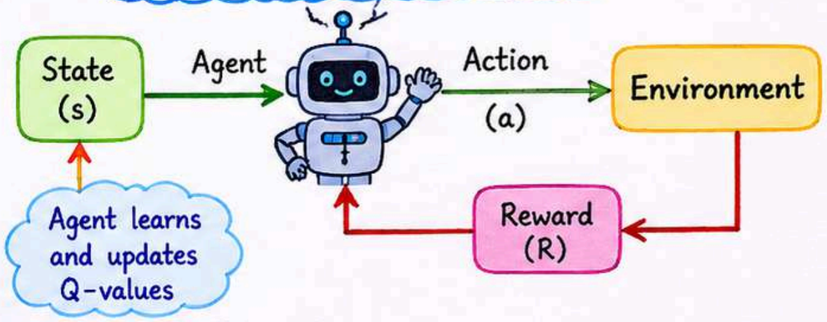
Features

1) Table-based :
   - It uses a table to store Q-values for each state-action pair Q(s,a).
2) Learning from Experience :
   - The agent explores the environment and updates Q-values using the rule:

$$Q(s,a) = Q(s,a) + \alpha \left[R + \gamma \max_{a'} Q(s',a') - Q(s,a) \right]$$

- α = learning rate γ = discount factor
- R = reward received
- $\max Q(s',a')$ = max future reward from next state

How Q-Learning Works

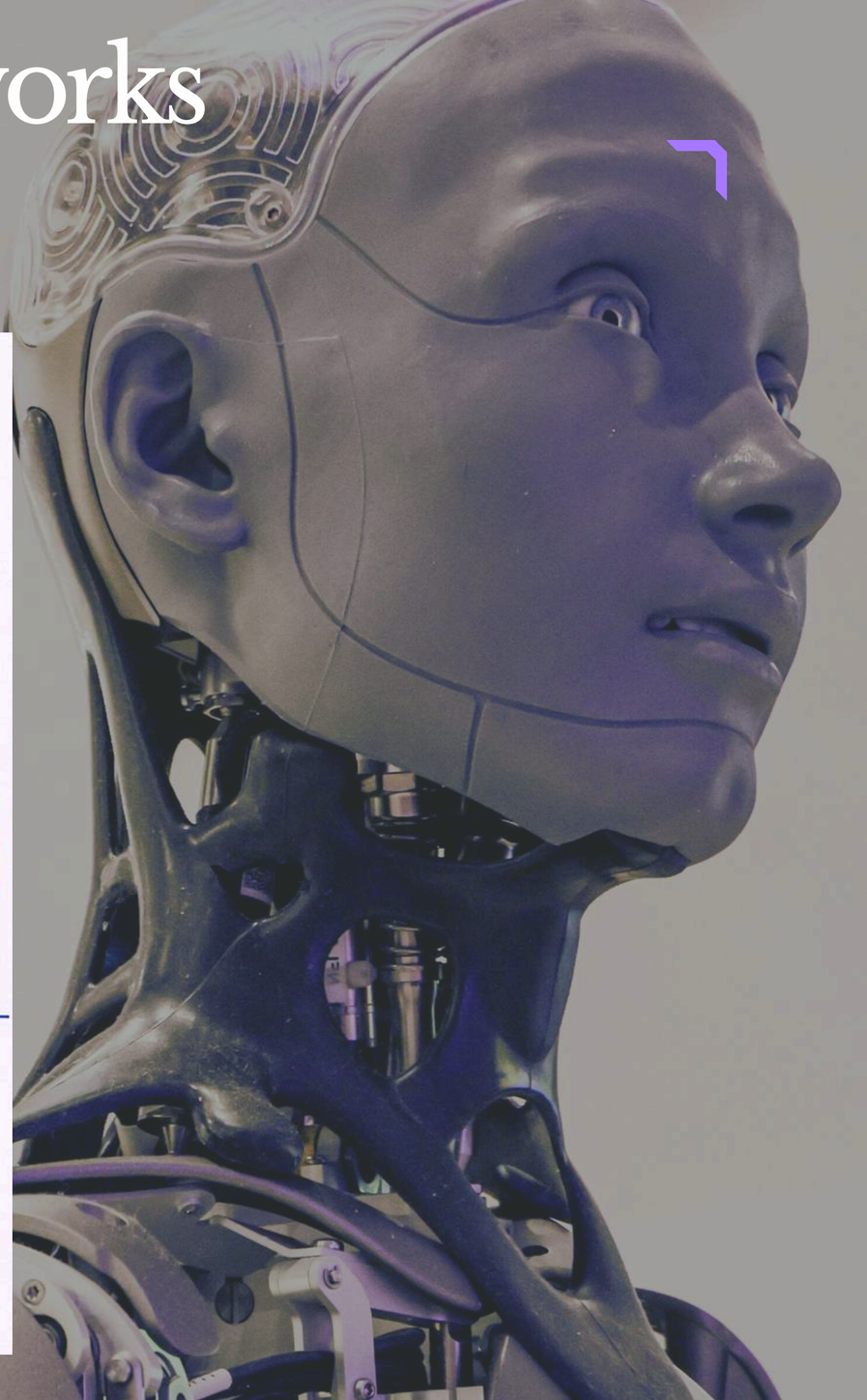


Q-Table Example :

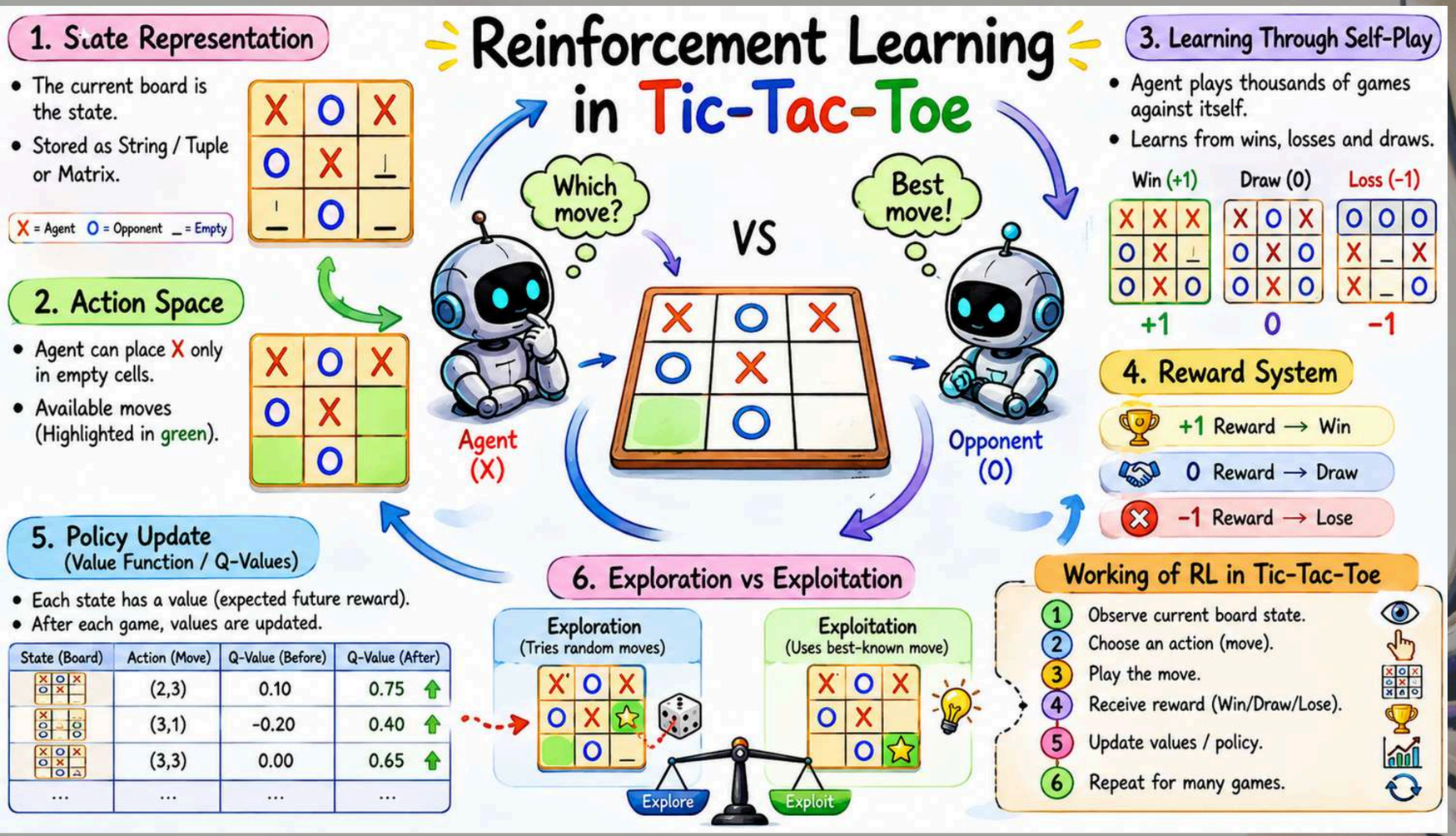
State	Actions			
	Left	Right	Up	Down
S_1	5.2	7.1	3.4	2.1
S_2	1.3	6.5	4.8	2.0
...	...	...	...	...

Higher Q-value means better action.

3) Exploration vs Exploitation :
   - The agent uses ε-greedy strategy.

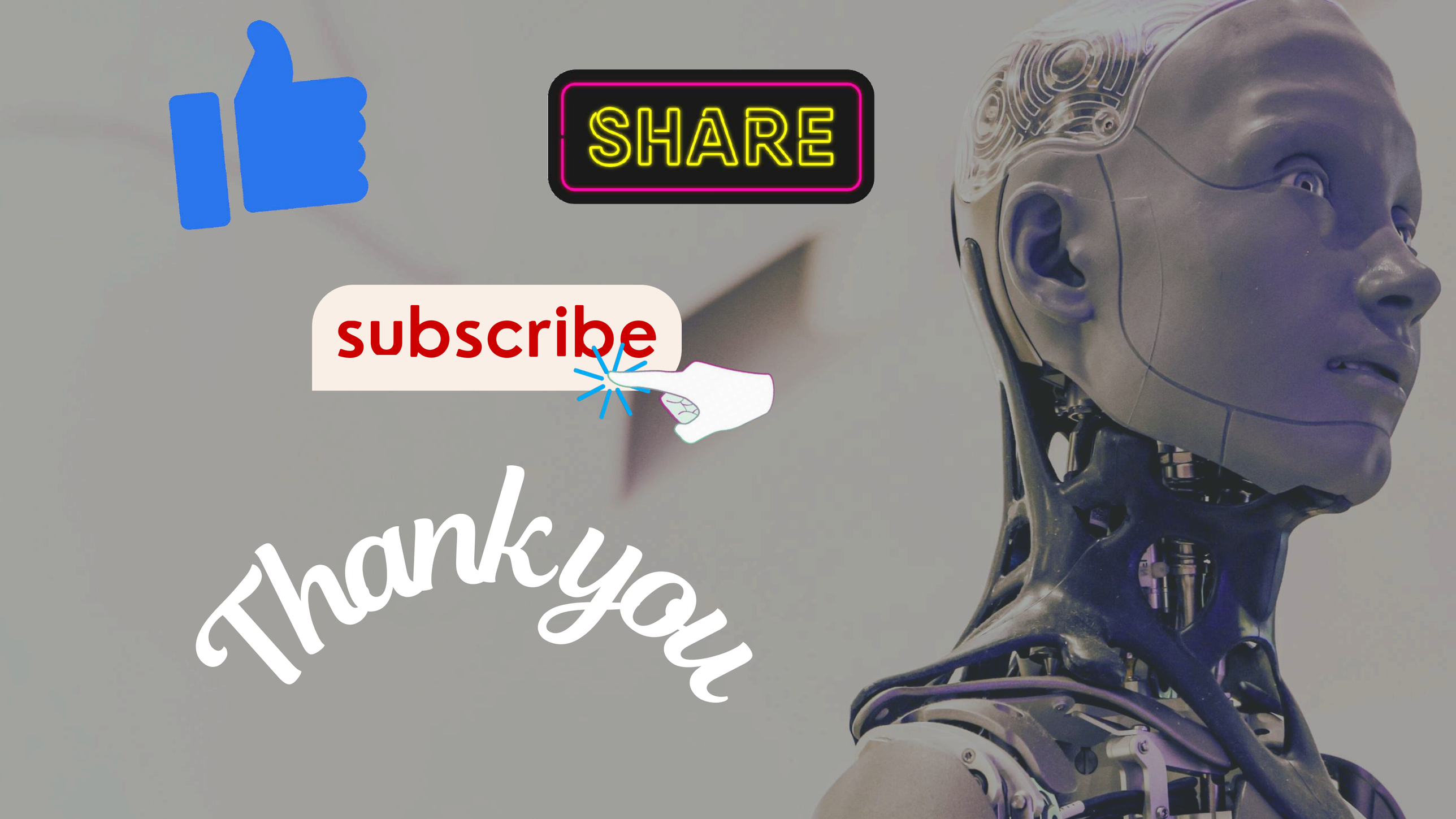


Dynamic Programming in RL



Difference

Active Reinforcement Learning	Passive Reinforcement Learning
The agent learns **which action to take** in each state.	The agent follows a **fixed policy** already given.
Goal is to find the **optimal policy**.	Goal is to evaluate **how good the given policy is**.
Agent explores different actions.	Agent does not explore new actions.
Learning involves both **policy and value function**.	Learning mainly focuses on **value function**.
More complex and computationally expensive.	Simpler and easier to implement.
Uses exploration vs exploitation strategy.	No exploration is needed.
Example: Self-driving car learning best driving strategy.	Example: Robot following predefined instructions and learning outcomes.


SHARE
subscribe
Thankyou